

Informe Final del Sistema

Documentación técnica, decisiones de diseño, pruebas y transferencia de conocimiento

Equipo de desarrollo

Martín Ramírez Espinosa

28 de mayo de 2026

Índice general

Nota para los autores	6
0.1. Introducción general	7
0.2. Análisis de requerimientos, restricciones y uso	7
0.3. Diseño general del sistema	8
0.4. Hardware	8
0.4.1. Propósito de la sección	8
0.4.2. Contexto dentro del sistema general	8
0.4.3. Desarrollo técnico	11
0.4.4. Entradas, salidas e interfaces	15
0.4.5. Decisiones de diseño o implementación	15
0.4.6. Problemas presentados y ajustes realizados	16
0.4.7. Validación, pruebas o evidencias	17
0.4.8. Aprendizajes y transferencia de conocimiento	17
0.4.9. Limitaciones	18
0.4.10. Recomendaciones específicas	18
0.5. Software del sensor	18
0.5.1. Resumen	18
0.5.2. Introducción y arquitectura general	19
0.5.3. Bloques o partes del código	20
0.5.4. Uso de buffers	21
0.5.5. Relación con el backend	22

0.5.6.	Balance IQ	22
0.5.7.	Correccion de componente DC	22
0.5.8.	Estimacion de PSD	23
0.5.9.	Demodulacion FM y AM	23
0.5.10.	Filtrado	24
0.5.11.	Trabajo multihilo	24
0.5.12.	Remocion del pico DC en PSD	25
0.5.13.	Calibracion en frecuencia	25
0.5.14.	Orquestador	26
0.5.15.	Campanas	27
0.5.16.	PSD en tiempo real	27
0.5.17.	PSD y audio en tiempo real	27
0.5.18.	Calibracion	27
0.5.19.	Protocolos y procedimientos nombrados	28
0.5.20.	Flujo completo de funcionamiento	28
0.6.	Plataforma web en la nube	29
0.7.	Análisis espectral y localización en la nube	29
0.7.1.	Propósito de la sección	29
0.7.2.	Contexto dentro del sistema general	30
0.7.3.	Arquitectura del módulo de postprocesamiento	30
0.7.4.	Flujo general de procesamiento	32
0.7.5.	Entradas del sistema	32
0.7.6.	Salidas del sistema	34
0.7.7.	Modos de operación	35
0.7.8.	Análisis espectral realizado: frecuencia central, ancho de banda, potencia y MER/BER	37
0.8.	Protocolos de prueba en nube y borde	41
0.9.	Integración general del sistema	41
0.10.	Conclusiones generales	42

0.11. Recomendaciones	43
0.12. Referencias	43
A. Técnica de desarrollo en grupo usando agentes IA	44
A.1. Propósito del capítulo	44
A.2. Contexto dentro del sistema general	44
A.3. Desarrollo técnico: Ecosistema de Agentes CLI	44
A.3.1. Instalación y Configuración	45
A.3.2. Arquitectura Multi-Agente	45
A.4. Decisiones de diseño: Metodología de Prompting	47
A.5. Problemas presentados y ajustes realizados	47
A.6. Validación y Evidencias	47
A.7. Aprendizajes y transferencia de conocimiento	48
A.8. Recomendaciones específicas	48
A.9. Diagramas adicionales	49
A.10. Fragmentos de código relevantes	49
A.11. Manuales o guías de uso	50
A.12. Resultados de pruebas	50

Índice de figuras

1.	Vista del conjunto físico del nodo ANE2. Fuente: elaboración propia. . . .	10
2.	Módulos principales del hardware ANE2. Fuente: elaboración propia. . . .	12
3.	Detalle esquemático de rutas RF, filtros y etapa tipo mux/switch en Mon-RaF2. Fuente: elaboración propia.	13
A.1.	Diseño conceptual del sistema multi-agente: Relación entre agentes y el flujo de decisión. Fuente: Elaboración propia.	46
A.2.	Roles asignados y estados de diseño en el flujo de trabajo IA. Fuente: Elaboración propia.	48

Índice de tablas

1.	Inventario funcional de componentes de hardware del nodo ANE2.	11
2.	Criterios mínimos para una versión robusta del árbol de potencia.	14
3.	Interfaces principales del hardware ANE2.	15
4.	Decisiones de diseño de hardware y justificación.	15
5.	Problemas presentados, decisiones y ajustes realizados en hardware.	16
6.	Resumen de telemetría eléctrica y térmica inspeccionada.	17
8.	Archivos principales del módulo de postprocesamiento espectral.	31
9.	Entradas principales consideradas en este avance del módulo.	33
10.	Salidas generales del procesamiento.	34
11.	Campos principales dentro de cada resultado.	34
12.	Modos de operación del módulo de postprocesamiento.	36
13.	Campos asociados a la frecuencia central.	38
14.	Campos asociados al ancho de banda.	39
15.	Campos asociados a potencia.	40
16.	Campos asociados a MER/BER cuando la estimación aplica.	41
A.1.	Componentes de un Prompt de alta calidad.	47
A.2.	Problemas en la colaboración con IA y ajustes aplicados.	47

Nota para los autores

Este documento corresponde al **informe final del sistema**. El archivo `main.tex` es el archivo maestro y debe ser modificado únicamente por el integrador del documento.

Cada autor debe trabajar únicamente en el archivo correspondiente a su capítulo dentro de la carpeta `section/`. La estructura, reglas de escritura y criterios de calidad se encuentran en el archivo `README.md`.

Cada capítulo técnico debe documentar:

- Propósito del capítulo.
- Contexto dentro del sistema general.
- Desarrollo técnico.
- Entradas, salidas e interfaces.
- Decisiones de diseño o implementación.
- Problemas presentados y ajustes realizados.
- Validación, pruebas o evidencias.
- Aprendizajes y transferencia de conocimiento.
- Limitaciones.
- Recomendaciones específicas.

Si se menciona software, el código fuente completo no debe incluirse en este informe. Debe explicarse la función del software y enlazarse el repositorio real correspondiente. Si el repositorio no existe todavía, debe escribirse explícitamente: *Repositorio pendiente de publicación*.

Antes de la versión final no debe quedar ninguna marca `\pendiente{}` en el documento.

0.1. Introducción general

Esta sección debe presentar el contexto del proyecto, el problema que se busca resolver y la motivación del sistema. También sirve para explicar, de forma breve, cuál es el alcance del documento y cómo se relacionan sus partes principales.

Como placeholder, este texto puede reemplazarse posteriormente por una narrativa más formal que describa el objetivo del sistema, los actores involucrados y el valor técnico de la solución propuesta.

Puntos sugeridos para completar

- Descripción del problema o necesidad principal.
- Objetivo general del sistema.
- Alcance del documento técnico.
- Resumen corto de la arquitectura o módulos principales.
- Relación entre borde, nube y análisis posterior.

0.2. Análisis de requerimientos, restricciones y uso

Esta sección debe consolidar los requerimientos funcionales y no funcionales del sistema, así como las restricciones de hardware, software, conectividad, presupuesto, tiempo o entorno de despliegue que condicionan el diseño.

El placeholder actual deja definido el espacio donde luego podrán incluirse tablas, listas o descripciones de casos de uso que permitan justificar decisiones de arquitectura y priorización técnica.

Puntos sugeridos para completar

- Requerimientos funcionales del sistema.
- Requerimientos no funcionales: latencia, disponibilidad, seguridad, escalabilidad.
- Restricciones de adquisición, procesamiento y transmisión de datos.
- Limitaciones del entorno operativo o experimental.
- Casos de uso o escenarios de operación.

0.3. Diseño general del sistema

En esta sección debe describirse la arquitectura general del sistema y la forma en que interactúan sus componentes. El objetivo es mostrar una visión integrada del flujo de datos, las interfaces entre módulos y la justificación de las decisiones de diseño.

Por ahora, este bloque funciona como placeholder para incorporar diagramas, descripciones de componentes, contratos de comunicación y criterios de integración entre borde y nube.

Puntos sugeridos para completar

- Arquitectura general y diagrama de alto nivel.
- Componentes principales y sus responsabilidades.
- Flujo de datos entre sensores, procesamiento en borde y servicios en nube.
- Interfaces, protocolos o formatos de intercambio.
- Decisiones de diseño y trade-offs técnicos.

0.4. Hardware

0.4.1. Propósito de la sección

Esta sección documenta el hardware del nodo de sensado espectral ANE2 con un criterio de transferencia de conocimiento. La intención no es solamente listar piezas, sino explicar cómo se organiza el montaje físico, qué función cumple cada módulo, cuáles interfaces deben respetarse y qué decisiones deben mantenerse en una nueva versión del producto.

La idea técnica central es la alimentación. La experiencia de integración mostró que concentrar la entrega de energía de los periféricos en la Raspberry Pi 5 reduce el margen operativo del nodo. Un diseño derivado o una versión posterior debe incluir un **árbol de potencia externo, sobredimensionado y medible**, capaz de alimentar cómputo, SDR, LTE/GNSS, mux RF y futuras extensiones sin depender del margen de los puertos de la Raspberry Pi.

0.4.2. Contexto dentro del sistema general

ANE2 es un nodo de borde para monitoreo espectral. En campo, el hardware debe recibir señales RF, seleccionar o enrutar la entrada de antena, digitalizar la señal mediante SDR,

ejecutar procesamiento local, obtener ubicación y mantener conectividad con la plataforma. Por tanto, el hardware cumple cuatro responsabilidades:

- **Entrada y selección RF:** las antenas llegan a la tarjeta MonRaF2, donde se concentran rutas RF, filtros y una etapa tipo mux/switch controlada por GPIO.
- **Adquisición SDR:** el HackRF One opera como receptor y entrega muestras IQ al computador de borde.
- **Cómputo de borde:** la Raspberry Pi 5 ejecuta control, telemetría, adquisición, procesamiento y comunicación con servicios superiores.
- **Conectividad y ubicación:** el módulo SIM7600G-H aporta LTE y GNSS/GPS para comunicación y georreferenciación.

La Figura 1 presenta el conjunto físico del nodo. La lectura correcta del montaje es: antenas hacia MonRaF2, selección o acondicionamiento RF en MonRaF2, adquisición con HackRF One, procesamiento en Raspberry Pi 5 y comunicación por LTE/GNSS.

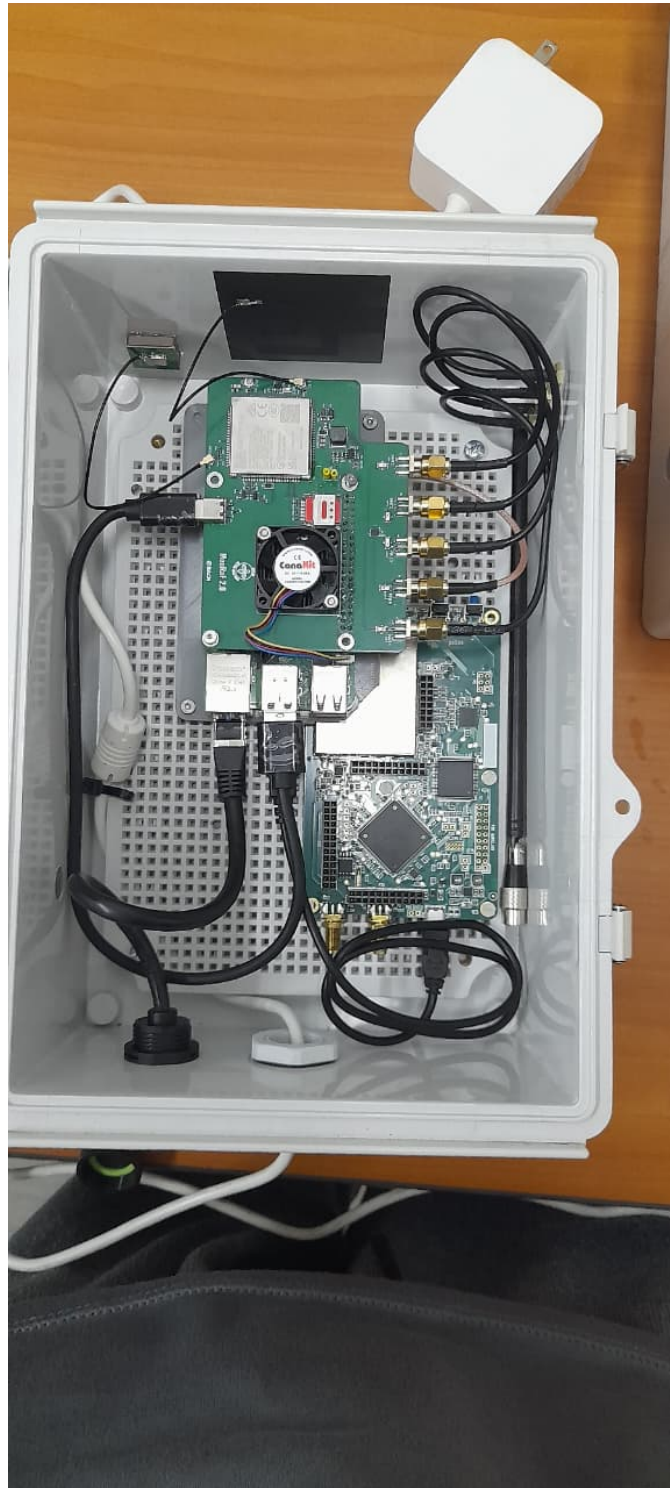


Figura 1: Vista del conjunto físico del nodo ANE2. Fuente: elaboración propia.

0.4.3. Desarrollo técnico

Arquitectura física del nodo

El nodo se compone de una Raspberry Pi 5, una tarjeta HackRF One y una PCB MonRaF2. La Raspberry Pi 5 coordina la ejecución de software, el HackRF One realiza la recepción SDR y MonRaF2 concentra conectividad, geolocalización, señales de control, protecciones y rutas RF.

Tabla 1: Inventario funcional de componentes de hardware del nodo ANE2.

Componente	Función principal	Criterio de integración
Raspberry Pi 5	Computador de borde	Ejecuta control, telemetría, adquisición, procesamiento y comunicación. No debe tratarse como fuente principal para todos los periféricos en una versión robusta.
HackRF One	Receptor SDR	Digitaliza señales RF como muestras IQ. Debe recibir la ruta RF ya seleccionada/acondicionada y requiere una alimentación estable para evitar fallas bajo carga.
MonRaF2	Tarjeta de integración	Integra LTE/GNSS, UART/USB/GPIO, protecciones, filtros y una ruta RF con función tipo mux/switch. Es el punto de terminación de antenas y control de rutas RF.
SIM7600G-H	LTE/GNSS	Proporciona datos celulares y ubicación. Su consumo y transitorios deben considerarse dentro del árbol de potencia.
Antenas RF, LTE y GNSS	Interfaz electromagnética	Se conectan al subsistema de MonRaF2. La conexión no debe documentarse como antena directa al HackRF, porque el diseño incluye una etapa de selección/acondicionamiento RF.
Árbol de potencia externo	Distribución de energía	Debe proveer rieles y margen para Raspberry Pi 5, HackRF, MonRaF2, LTE/GNSS y extensiones futuras. Es un requisito de diseño, no una mejora opcional.

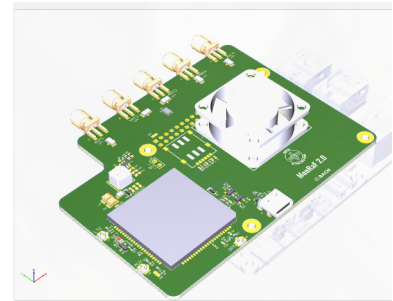
La Figura 2 muestra los tres módulos principales del nodo. Su separación ayuda a diagnosticar fallas: no es lo mismo un problema de ruta RF, de alimentación, de procesamiento, de enlace celular o de ubicación.



(a) Raspberry Pi 5.



(b) HackRF One.



(c) MonRaF2.

Figura 2: Módulos principales del hardware ANE2. Fuente: elaboración propia.

Ruta RF y mux de antenas

La ruta RF no debe describirse como una antena conectada directamente al HackRF. El esquemático de MonRaF2 muestra redes de antena como MAIN_ANT, GNSS_ANT y AUX_ANT, además de conectores RF, filtros LPF y una etapa de conmutación/mux RF controlada por señales GPIO. En términos operativos, MonRaF2 actúa como la tarjeta que recibe y ordena las rutas de antena antes de que la señal sea usada por el subsistema de adquisición o por el módulo celular/GNSS.

El flujo físico recomendado queda entonces:

1. Las antenas externas se conectan a MonRaF2.
2. MonRaF2 selecciona, protege o acondiciona rutas RF mediante su red de filtros, conectores y mux/switch RF.
3. La ruta RF seleccionada alimenta el subsistema correspondiente: adquisición SDR, LTE o GNSS, según el modo de operación.
4. La Raspberry Pi 5 controla la operación y recibe los datos del HackRF por USB, pero no debe asumirse como concentrador físico de antenas.

- Las pruebas con alimentación externa para periféricos tuvieron mejor margen para sostener carga combinada.

Por ello, el criterio para un nuevo producto o una siguiente versión debe ser:

El nodo debe tener un árbol de potencia externo y generoso, con capacidad para la carga actual y para extensiones futuras. La Raspberry Pi 5 debe ser tratada como carga y controlador, no como fuente primaria para el resto del sistema.

Tabla 2: Criterios mínimos para una versión robusta del árbol de potencia.

Criterio	Implicación de diseño
Fuente principal sobredimensionada	Debe alimentar simultáneamente Raspberry Pi 5, HackRF, Mon-RaF2, LTE/GNSS y periféricos futuros sin operar en el límite.
Rieles distribuidos	Las cargas de alto consumo o con transitorios, como SDR y LTE, deben alimentarse desde rieles dedicados o reguladores con margen, no desde el puerto de la Raspberry Pi.
Medición de rieles	El sistema debe conservar telemetría de voltaje, corriente, temperatura y eventos de subtensión para diagnóstico.
Protección y desacople	Se deben incluir protecciones, filtrado y capacitancia local cerca de cargas ruidosas o pulsantes.
Reserva para extensiones	El diseño debe contemplar nuevos servicios, antenas, sensores, almacenamiento o comunicaciones sin rediseñar la alimentación desde cero.

0.4.4. Entradas, salidas e interfaces

Tabla 3: Interfaces principales del hardware ANE2.

Interfaz	Tipo	Contrato operacional
Antenas MonRaF2	a RF coaxial	Las antenas se conectan a MonRaF2. La etapa RF de la tarjeta selecciona o acondiciona rutas antes de la adquisición o uso LTE/GNSS.
MonRaF2 HackRF	a RF seleccionado	El HackRF debe recibir una ruta RF controlada, no una conexión de antena asumida como directa.
HackRF Raspberry Pi 5	a USB	Transporta muestras IQ y comandos de configuración. El <i>sample rate</i> aumenta simultáneamente el ancho instantáneo y la presión sobre USB/CPU.
Raspberry Pi 5 a MonRaF2	GPIO, UART, USB	Permite control auxiliar, comunicación con LTE/GNSS y señales de estado.
MonRaF2 red celular	a LTE	Provee conectividad de datos para estado, mediciones, campañas y reintentos.
MonRaF2 GNSS	a GNSS/GPS	Entrega ubicación del nodo para reportes y trazabilidad de mediciones.
Fuente externa a cargas	DC regulado	Debe distribuir potencia a Raspberry Pi 5, HackRF, MonRaF2 y expansiones con margen suficiente.

0.4.5. Decisiones de diseño o implementación

Tabla 4: Decisiones de diseño de hardware y justificación.

Decisión	Justificación	Implicación práctica
Usar MonRaF2 como concentrador RF y de conectividad	La tarjeta contiene rutas de antena, filtros, mux/switch RF, LTE/GNSS y señales de control.	Las antenas se documentan y cablean hacia MonRaF2; el HackRF queda como receptor SDR dentro de la cadena, no como punto directo de antena.
Usar Raspberry Pi 5 como computador de borde	Tiene capacidad suficiente para orquestación, telemetría y control del SDR.	Debe protegerse de sobrecarga eléctrica; no debe alimentar todos los periféricos en una versión robusta.

Decisión	Justificación	Implicación práctica
Usar HackRF One como receptor SDR	Permite recepción flexible por software y adquisición IQ.	Requiere ganancia configurada con criterio y alimentación estable.
Separar potencia de control	Los problemas de sub-tensión aparecen cuando la distribución de energía queda demasiado acoplada a la Raspberry Pi.	El árbol de potencia externo pasa a ser requisito de producto.
Mantener telemetría de salud	Temperatura, voltajes, corrientes y banderas de <i>throttling</i> permiten diagnosticar fallas reales.	Cada prueba de RF o conectividad debe acompañarse de telemetría eléctrica.

0.4.6. Problemas presentados y ajustes realizados

Tabla 5: Problemas presentados, decisiones y ajustes realizados en hardware.

Problema	Evidencia	Causa probable	Decisión o ajuste	Resultado
Subtensión y bajo margen de potencia	Caídas de EXT5V bajo cargas SDR/LTE y escenarios con eventos de alimentación insuficiente.	Demasiadas cargas alimentadas o condicionadas desde la Raspberry Pi 5.	Diseñar un árbol de potencia externo, generoso y medible.	Requisito obligatorio para una versión robusta.
Aumento de temperatura bajo carga	Pruebas con HackRF y peticiones desde servidor alcanzaron temperaturas cercanas a 75 °C.	Procesamiento sostenido, actividad USB y carga RF/LTE simultánea.	Mantener disipación, ventilación y telemetría de temperatura.	La temperatura queda como restricción operativa.
Ambigüedad en ruta de antenas	El diseño incluye redes MAIN/GNSS/AUX, filtros y mux/switch RF en MonRaF2.	Interpretar la antena como conexión directa al HackRF simplifica de forma incorrecta la cadena RF.	Documentar MonRaF2 como entrada y selección RF.	La conexión física queda alineada con el esquemático.
Dependencia del enlace LTE	Las pruebas de campo usan conectividad celular para operación y envío de datos.	La red celular puede variar por cobertura, antena, SIM y ubicación.	Separar diagnóstico de RF, cómputo, potencia y LTE.	Las fallas se pueden atribuir con mejor evidencia.
Riesgo de saturación RF	HackRF no usa AGC de hardware en esta cadena de operación.	Ganancia excesiva o emisores fuertes pueden saturar el ADC de 8 bits.	Ajustar ganancia por escenario y conservar configuraciones durante comparaciones.	Menor riesgo de confundir artefactos con emisiones reales.

0.4.7. Validación, pruebas o evidencias

La validación usada para esta sección combina inspección de esquemático, fotografías del montaje y telemetría de nodos bajo reposo, carga SDR, carga LTE y carga combinada. Los indicadores más útiles fueron temperatura ARM, EXT5V, corrientes de rieles, banderas de subtensión, frecuencia de CPU y estados de *throttling*.

Tabla 6: Resumen de telemetría eléctrica y térmica inspeccionada.

Escenario	Temperatura ARM	EXT5V	UV	Lectura para operación
Nodo en reposo	54.3–57.6 °C	5.006–5.041 V	No	Referencia base con sistema operativo y telemetría.
HackRF en carga alta	54.9–74.7 °C	4.748–4.965 V	No	La carga SDR aumenta demanda y temperatura.
Periféricos conectados en reposo	45.0–52.1 °C	4.903–4.963 V	No	El margen mejora cuando no hay procesamiento pesado.
Petición media con periféricos	61.5–75.7 °C	4.721–5.083 V	No	El sistema se acerca a límites térmicos y de alimentación.
Carga alta con periféricos externos	49.4–75.2 °C	4.904–5.122 V	No	La alimentación externa mejora el margen frente a carga combinada.

0.4.8. Aprendizajes y transferencia de conocimiento

- El árbol de potencia define la robustez del nodo. Si se alimentan periféricos exigentes desde la Raspberry Pi, el diseño queda sin margen.
- Las antenas pertenecen a la cadena de MonRaF2 y su etapa RF. El HackRF debe verse como receptor dentro de una ruta seleccionada, no como destino directo de todas las antenas.
- La telemetría eléctrica debe acompañar cualquier prueba RF. Sin voltajes, corrientes y temperatura es fácil confundir fallas de potencia con fallas de software.
- El diseño debe reservar margen para servicios futuros. Cada extensión de software o hardware aumenta demanda de energía, temperatura y tráfico.

0.4.9. Limitaciones

- Las cifras de telemetría resumen pruebas disponibles; no reemplazan una caracterización eléctrica completa con instrumentos de laboratorio.
- No se presentan curvas completas de consumo por banda, ganancia, temperatura ambiente, antena o ubicación.
- El diseño RF requiere validar pérdidas, aislamiento, adaptación de impedancias y compatibilidad electromagnética antes de una versión final de campo.
- La estabilidad LTE/GNSS depende de cobertura, antenas, SIM, ubicación y configuración del operador.

0.4.10. Recomendaciones específicas

1. Diseñar una fuente principal externa con margen amplio para carga simultánea y extensiones futuras.
2. Alimentar HackRF, LTE/GNSS y cargas auxiliares desde rieles dedicados o reguladores adecuados, no desde el margen de la Raspberry Pi 5.
3. Mantener medición de voltaje, corriente, temperatura y banderas de subtenión como parte del producto.
4. Conectar y documentar antenas desde MonRaF2 y su ruta tipo mux/switch RF.
5. Revisar disipación térmica para escenarios con SDR y LTE activos al mismo tiempo.
6. Separar pruebas de reposo, SDR, LTE/GNSS y carga combinada para poder atribuir fallas con evidencia.

0.5. Software del sensor

0.5.1. Resumen

Este documento describe, de forma pedagógica y conceptual, la arquitectura del código que corre directamente en el sensor de monitoreo espectral. El sensor está compuesto por una Raspberry Pi 5, una HackRF One y una tarjeta auxiliar que integra comunicación LTE, receptor GPS y un multiplexor para selección de antena.

La sección del proyecto descrita aquí no corresponde a toda la plataforma. El sistema completo también incluye componentes externos, como backend, frontend, bases de datos y servicios de visualización. Este repositorio se concentra en el código embebido del sensor:

la parte que recibe instrucciones de la plataforma, controla el hardware de radiofrecuencia, adquiere muestras, procesa senales y devuelve resultados.

Proposito general

El proposito general del codigo es convertir al sensor fisico en un nodo autonomo de medicion espectral. Para ello, el software debe recibir ordenes del servidor, configurar la HackRF One, seleccionar la antena adecuada, capturar senales IQ, procesarlas localmente y enviar a la plataforma central productos utiles como PSD, metricas de modulacion, audio demodulado, estado del dispositivo y coordenadas GPS.

Objetivos especificos

- Controlar la adquisicion de radio mediante la HackRF One, ajustando frecuencia central, tasa de muestreo, ganancias, amplificador y puerto de antena.
- Ejecutar mediciones espectrales en modo *realtime* y en campanas programadas desde el servidor.
- Procesar muestras IQ crudas para estimar densidad espectral de potencia mediante metodos como Welch y banco de filtros polifase.
- Demodular senales AM o FM cuando el servidor solicita audio en tiempo real.
- Aplicar etapas de limpieza y correccion para mejorar la calidad de las mediciones, incluyendo balance IQ y remocion del pico DC.
- Mantener una calibracion de frecuencia que compense desviaciones del oscilador de la HackRF.
- Coordinar la comunicacion con el backend, el envio de resultados, los reintentos ante fallos de red y el reporte de estado del sensor.
- Gestionar recursos fisicos del sensor, como LTE, GPS, GPIO y multiplexor de antena.

0.5.2. Introduccion y arquitectura general

La arquitectura del sensor se puede entender como una cadena de trabajo: el servidor decide que se debe medir, el orquestador traduce esa instruccion a una configuracion de radio, el motor RF adquiere y procesa las muestras, y finalmente los resultados regresan al servidor.

La division principal del sistema es la siguiente:

- **Orquestacion en Python:** gestiona la relacion con el backend, consulta instrucciones, programa campanas, controla estados del sistema, envia comandos al motor RF y prepara los resultados para publicarlos.
- **Motor RF en C:** controla la HackRF One, recibe muestras IQ, administra buffers de alta velocidad, ejecuta procesamiento DSP, calcula PSD, demodula audio y devuelve resultados al orquestador.
- **Servicios auxiliares:** atienden funciones de soporte como GPS, LTE, estado del sistema, reintentos de datos no enviados, WebRTC para audio y control del multiplexor de antena.

La decision de separar Python y C responde a una necesidad practica. Python es adecuado para coordinacion, comunicacion HTTP, manejo de estados y programacion de tareas. C es adecuado para la parte sensible a latencia: captura de muestras, buffers, FFT, filtrado, PSD y demodulacion.

Servicios y modos de operacion

El sensor opera con varios servicios o modos complementarios:

- **PSD en tiempo real:** el backend solicita una configuracion y el sensor responde con una estimacion espectral de la banda capturada.
- **PSD y audio en tiempo real:** ademas de la PSD, el sensor demodula FM o AM y genera tramas de audio reproducibles por la plataforma.
- **Campanas:** el backend programa mediciones periodicas; el sensor guarda la configuracion, agenda ejecuciones y sube resultados en cada intervalo.
- **Calibracion en frecuencia:** el sensor estima el error de sintonia y aplica una correccion para que las mediciones queden mejor alineadas en frecuencia.
- **GPS y LTE:** el sensor mantiene conectividad celular y reporta ubicacion geografica al servidor.
- **Estado del dispositivo:** se reportan variables de salud como conectividad, almacenamiento, temperatura, memoria y actividad del software.

0.5.3. Bloques o partes del codigo

El codigo puede leerse como un conjunto de bloques funcionales. Cada bloque cumple una responsabilidad dentro del camino completo de la senal: desde la instruccion del backend hasta la publicacion del resultado.

Adquisicion

El bloque de adquisicion es el encargado de hablar con la HackRF One y convertir una instruccion de medicion en muestras IQ reales.

Conceptualmente, este bloque realiza cinco acciones:

1. Recibe una configuracion de medicion enviada por el orquestador. Esa configuracion contiene parametros como frecuencia central, ancho de muestreo, ganancias, metodo de PSD, puerto de antena, filtro solicitado y modo de demodulacion.
2. Aplica la configuracion sobre la HackRF One. Esto incluye sintonia de frecuencia, tasa de muestreo, ganancias de recepcion y correccion de frecuencia cuando existe una calibracion disponible.
3. Inicia la recepcion de muestras IQ crudas desde el dispositivo SDR.
4. Deposita esas muestras en buffers circulares para separar la velocidad de llegada del hardware de la velocidad de procesamiento del software.
5. Entrega bloques de muestras suficientemente grandes a los algoritmos de procesamiento, evitando que cada operacion pesada interrumpa directamente la captura.

Las muestras IQ representan la senal recibida en banda base compleja. La componente I corresponde a la parte en fase y la componente Q a la parte en cuadratura. En conjunto permiten conservar informacion de amplitud y fase, que es necesaria para estimar el espectro, filtrar, demodular FM o demodular AM.

0.5.4. Uso de buffers

La logica de buffers es central en el sensor. La HackRF entrega datos de forma continua y rapida, mientras que la PSD, el filtrado o la demodulacion requieren procesamiento por bloques. Por eso el motor RF usa buffers circulares:

- Un buffer principal recibe muestras IQ para estimacion espectral.
- Un buffer de audio conserva muestras necesarias cuando se activa demodulacion AM o FM.
- El software drena estos buffers cuando ya hay suficientes muestras para construir una respuesta.

Esta estructura permite que el sensor atienda procesos en tiempo real, como PSD y audio, sin tratar cada muestra de manera aislada. La idea pedagogica es similar a una banda transportadora: el hardware deposita muestras, el procesamiento toma paquetes completos y el orquestador recibe resultados ya elaborados.

0.5.5. Relacion con el backend

La adquisicion no decide por si sola que medir. Las instrucciones provienen del backend y llegan al motor RF a traves del orquestador. El backend define la intencion; el orquestador valida y traduce esa intencion; el motor RF la ejecuta sobre el hardware.

Este esquema permite que el sensor funcione como un nodo remoto controlado por la plataforma, pero conservando autonomia local para manejar errores, tiempos de espera, buffers, reconexiones y estados internos.

Preproceso

El preproceso agrupa las operaciones que preparan la senal antes de extraer productos finales. Su objetivo es reducir artefactos propios del hardware y mejorar la calidad de la medicion.

0.5.6. Balance IQ

En un receptor SDR real, las ramas I y Q no siempre quedan perfectamente balanceadas. Puede haber diferencias de ganancia, pequenos desplazamientos DC o correlacion no deseada entre ambas componentes. Si estos efectos no se corrigen, pueden aparecer artefactos en el espectro o degradacion en la demodulacion.

El balance IQ busca compensar esos errores antes del calculo de PSD o del filtrado. De forma conceptual, el algoritmo:

- remueve desplazamientos promedio en las ramas I y Q;
- compara la energia relativa de ambas ramas;
- corrige diferencias de amplitud;
- reduce correlaciones espurias entre I y Q.

No se trata de cambiar la informacion de la senal recibida, sino de disminuir imperfecciones introducidas por la cadena de recepcion.

0.5.7. Correccion de componente DC

La componente DC aparece cerca del centro de la banda base y suele estar asociada a efectos internos del receptor. En el flujo del sensor se atiende en dos niveles:

- En el preproceso IQ se reduce el offset de las muestras antes de calcular PSD o demodular.
- En el postproceso de PSD se repara el pico central visible en el espectro cuando aparece como un artefacto de medicion.

Esta separacion es importante: una cosa es limpiar la senal cruda y otra es pulir el producto espectral que se enviara al servidor.

Proceso

El bloque de proceso transforma muestras IQ en productos utiles para la plataforma. En este repositorio se implementan tres familias principales de procesamiento: estimacion de PSD, demodulacion de audio y filtrado de senales.

0.5.8. Estimacion de PSD

La PSD, o densidad espectral de potencia, permite observar como se distribuye la energia de la senal dentro de un rango de frecuencias. Es el producto principal para monitoreo espectral, porque permite identificar ocupacion de banda, niveles relativos de potencia, portadoras y posibles emisiones.

El sensor contempla metodos como:

- **Welch:** divide la senal en segmentos, aplica ventanas, calcula FFT y promedia resultados para obtener una estimacion estable del espectro.
- **Banco de filtros polifase:** mejora la separacion entre canales y ayuda a controlar fugas espectrales cuando se requiere una estimacion mas selectiva.

La PSD es una de las tareas mas costosas del sistema, porque requiere operar sobre grandes bloques de muestras y ejecutar transformadas de Fourier. Por eso el motor RF esta escrito en C y aprovecha una logica de trabajo multihilo y bibliotecas de procesamiento eficientes.

0.5.9. Demodulacion FM y AM

Cuando el backend solicita audio en tiempo real, el sensor no solo calcula PSD. Tambien toma las muestras IQ, extrae la informacion modulada y la convierte en audio reproducible.

En FM, la informacion esta principalmente en la variacion de fase o frecuencia instantanea. El proceso conceptual consiste en observar como cambia la fase entre muestras

consecutivas, convertir esa variación en una señal de audio y acondicionar el resultado para reproducción.

En AM, la información está principalmente en la envolvente de la señal. El proceso conceptual consiste en calcular la magnitud de la señal compleja, separar las variaciones útiles de la portadora y normalizar el audio resultante.

Después de demodular, el audio se convierte a un formato de transmisión. El sensor genera tramas de audio, las codifica con Opus y las entrega al servicio local que publica el flujo mediante WebRTC. Esto permite que la plataforma escuche la señal con baja latencia sin que el backend tenga que procesar IQ crudo.

0.5.10. Filtrado

El filtrado permite concentrarse en una parte de la banda capturada y rechazar contenido fuera del rango de interés. El servidor puede solicitar un rango de frecuencia y el motor RF aplica un filtro pasabanda antes de calcular la PSD o de preparar la señal.

Desde una perspectiva conceptual, el filtro funciona como una ventana selectiva: conserva la región que interesa y atenúa lo que queda por fuera. En el código se apoya en procedimientos de dominio frecuencial y validaciones para que el rango solicitado sea coherente con la banda realmente capturada por la HackRF.

0.5.11. Trabajo multihilo

El sistema separa tareas para mantener capacidad de respuesta:

- Un flujo atiende la llegada de muestras desde la HackRF.
- Otro flujo procesa bloques para PSD.
- Cuando hay audio, un hilo adicional drena muestras, demodula y envía tramas codificadas.
- El orquestador permanece disponible para comunicarse con el backend y manejar cambios de modo.

Esta organización evita que una tarea pesada, como una PSD de alta resolución, bloquee completamente la captura o la transmisión de audio.

Postproceso

El postproceso es la pulida final aplicada a los resultados antes de enviarlos al servidor. En este sistema, la etapa más importante es la corrección del pico DC en la PSD.

0.5.12. Remocion del pico DC en PSD

Aunque el preproceso reduce offset en las muestras IQ, en la PSD puede quedar un pico artificial cerca del centro de la banda. Ese pico no siempre representa una emision real; con frecuencia es un artefacto propio del receptor SDR.

El algoritmo de postproceso analiza el comportamiento estadistico de la PSD como una senal con variabilidad local. En lugar de borrar siempre un numero fijo de puntos, estima la region afectada y decide como reconstruirla.

Conceptualmente, el procedimiento realiza:

- deteccion de la region central afectada usando simetria y cambios locales en la pendiente;
- analisis de ocupacion espectral alrededor del centro;
- validaciones con descriptores estadisticos como media, mediana, dispersion y comportamiento de la cola alta;
- ampliacion de la ventana de reparacion cuando la zona tiene baja ocupacion espectral;
- reconstruccion local mediante interpolacion lineal o ajuste polinomial.

El resultado enviado al backend conserva una PSD mas limpia. Cuando aplica, el software tambien puede conservar informacion de diagnostico sobre la correccion, como la ventana reparada y el modo de reconstruccion usado.

0.5.13. Calibracion en frecuencia

La calibracion en frecuencia busca compensar la desviacion entre la frecuencia nominal solicitada y la frecuencia real a la que queda sintonizado el receptor. En SDR de bajo costo, esta desviacion puede aparecer por tolerancias del oscilador, temperatura o condiciones de operacion.

Idea principal

El sensor usa una referencia conocida para estimar el error de frecuencia. En el caso de emisiones FM comerciales, una referencia util es el tono piloto estereo de 19 kHz que aparece despues de demodular FM. Si una estacion FM fuerte contiene ese tono, el sensor puede usarlo como punto de comparacion.

El flujo conceptual es:

1. Capturar una porcion amplia de la banda donde se esperan emisoras FM.
2. Identificar candidatos con potencia suficiente.
3. Bajar cada candidato a banda base y demodular FM.
4. Buscar el tono piloto alrededor de 19 kHz en la senal demodulada.
5. Comparar la posicion observada con la posicion esperada.
6. Estimar un error de frecuencia y expresarlo como factor de correccion.

Uso de varias emisiones

Cuando hay varias emisiones candidatas, el sistema puede usar mas de una referencia para evitar depender de una sola estacion. Las emisiones mas confiables reciben mayor peso, por ejemplo si tienen mejor potencia o una deteccion mas clara del tono piloto. Con esa informacion se obtiene una estimacion mas robusta del error.

Aplicacion de la correccion

El resultado de la calibracion se guarda como un error de frecuencia, normalmente expresado en partes por millon. Luego, cuando el backend solicita una frecuencia central, el orquestador incluye ese factor en la configuracion enviada al motor RF.

El motor usa la correccion para ajustar la sintonia de la HackRF. De esta manera, la frecuencia fisica de adquisicion queda mas alineada con la frecuencia nominal que la plataforma espera visualizar.

0.5.14. Orquestador

El orquestador es la logica de coordinacion del sensor. Su funcion no es procesar IQ de forma intensiva, sino dirigir el trabajo de todos los bloques.

Responsabilidades principales

- Consultar al backend para saber si hay solicitudes en tiempo real.
- Consultar y sincronizar campanas programadas.
- Validar configuraciones antes de enviarlas al motor RF.
- Enviar comandos al motor RF usando mensajeria local ZeroMQ.

- Recibir resultados de PSD y metricas desde el motor RF.
- Aplicar postproceso sobre los resultados cuando corresponde.
- Formatear y publicar datos hacia el backend mediante HTTP REST.
- Controlar el inicio y parada del servicio de audio WebRTC.
- Mantener estados globales como `IDLE`, `REALTIME`, `CAMPAIGN`, `KALIBRATING` y `ERROR`.
- Gestionar memoria compartida local para parametros persistentes, como `ppm_error`, coordenadas GPS o configuraciones de campana.

Modos controlados por el orquestador

0.5.15. Campanas

Las campanas son mediciones programadas por el servidor. El orquestador consulta las campanas activas, selecciona la que corresponde ejecutar, guarda sus parametros y agenda ejecuciones periodicas. Cada ejecucion captura PSD y envia el resultado al backend. Si la red falla, el resultado puede quedar en una cola local para ser reenviado posteriormente.

0.5.16. PSD en tiempo real

En modo realtime, el servidor entrega una configuracion inmediata. El orquestador la envia al motor RF, espera la respuesta, aplica postproceso y sube el resultado. Mientras el backend siga enviando una solicitud valida, el sensor mantiene este modo activo.

0.5.17. PSD y audio en tiempo real

Cuando la solicitud incluye demodulacion, el orquestador activa tambien el camino de audio. En ese caso, el motor RF demodula AM o FM y genera tramas Opus; el servicio WebRTC se encarga de hacer disponible ese audio para la plataforma.

0.5.18. Calibracion

Antes o durante ciertos ciclos de operacion, el orquestador puede solicitar al motor RF una calibracion de frecuencia. El resultado se guarda localmente y se usa en adquisiciones posteriores.

0.5.19. Protocolos y procedimientos nombrados

El documento evita entrar en detalles de bajo nivel, pero es importante reconocer los protocolos y procedimientos principales que conectan los bloques:

Elemento	Uso conceptual en el sensor
HTTP REST	Comunicacion entre sensor y backend para consultar instrucciones y publicar datos.
ZeroMQ	Canal local de comandos entre el orquestador Python y el motor RF en C.
libhackrf/USB	Interfaz de control y adquisicion de muestras desde la HackRF One.
WebRTC	Entrega de audio demodulado hacia cliente/plataforma con baja latencia.
Opus	Codificacion de audio antes de enviarlo al servicio WebRTC.
GPIO	Control de lineas fisicas para multiplexor de antena y soporte de hardware.
PPP/LTE	Conectividad celular del sensor hacia la red.
GPS/NMEA	Obtencion y conversion de coordenadas geograficas.
Welch	Metodo de estimacion de PSD por segmentos y promediado.
Banco polifase	Metodo de estimacion espectral con mejor control de separacion entre canales.

0.5.20. Flujo completo de funcionamiento

De forma resumida, el funcionamiento del sensor puede leerse como el siguiente ciclo:

1. El sensor se identifica ante la plataforma mediante su direccion MAC y su estado local.
2. El orquestador consulta si hay una tarea realtime o una campana activa.
3. Si hay tarea, se construye una configuracion de radio y se envia al motor RF.
4. El motor RF configura la HackRF, selecciona antena si aplica y captura IQ.
5. Las muestras pasan por preproceso para mejorar su condicion de medicion.
6. El bloque de proceso calcula PSD, aplica filtros y, si se solicita, demodula audio.

7. El postproceso corrige artefactos finales, especialmente el pico DC en la PSD.
8. El orquestador empaqueta el resultado y lo envía al backend.
9. Si el envío falla, el dato puede quedar almacenado localmente para reintento.
10. Los servicios auxiliares mantienen GPS, LTE, estado del sensor y audio según el modo activo.

0.6. Plataforma web en la nube

Esta sección debe describir la plataforma desplegada en la nube, incluyendo servicios, componentes web, almacenamiento, autenticación, observabilidad y mecanismos de despliegue o actualización de la aplicación.

El contenido placeholder deja lista una base para documentar la arquitectura web, la gestión de usuarios, la integración con datos provenientes del borde y los criterios de operación del sistema en un entorno remoto.

Puntos sugeridos para completar

- Arquitectura de la aplicación web y servicios en nube.
- Modelo de datos y persistencia.
- API, paneles o interfaces de usuario.
- Estrategias de seguridad, monitoreo y despliegue.
- Integración con el pipeline de adquisición y análisis.

0.7. Análisis espectral y localización en la nube

0.7.1. Propósito de la sección

Esta sección documenta el módulo de postprocesamiento espectral desarrollado para analizar, en un entorno centralizado, las capturas generadas por el sistema de monitoreo. El objetivo de este avance es explicar cómo el software recibe una traza espectral, organiza la información de frecuencia y amplitud, selecciona un modo de operación y obtiene los primeros parámetros técnicos de las emisiones detectadas.

En esta etapa se describe el proceso hasta la estimación de la **frecuencia central**, el **ancho de banda**, la **potencia** y, cuando la captura corresponde a un caso aplicable, **MER/BER**. La verificación detallada de cumplimiento contra licencias, el uso formal de DANE/municipio, la localización administrativa, las pruebas finales y las recomendaciones quedan como continuación de esta misma sección.

El propósito del módulo no es reemplazar la adquisición SDR realizada en el borde, sino convertir las capturas recibidas en resultados interpretables para la plataforma. Por esta razón, el capítulo se enfoca en la función del software dentro del flujo general del sistema, en sus entradas y salidas principales, y en las decisiones iniciales usadas para diferenciar ruido, picos solicitados y emisiones candidatas.

0.7.2. Contexto dentro del sistema general

El sistema completo puede entenderse como una cadena de sensado, transmisión, procesamiento y visualización. En el borde, el nodo de monitoreo recibe señales de radiofrecuencia mediante el hardware SDR y genera una captura espectral. Posteriormente, esa captura se envía hacia la plataforma central o se analiza de forma local mediante archivos de entrada. El módulo descrito en esta sección se ubica después de la adquisición y antes de la presentación de resultados al usuario.

La información que llega al módulo corresponde principalmente a una serie de muestras espectrales asociadas a un rango de frecuencias. Con esos datos, el software reconstruye el eje de frecuencia, identifica regiones de interés y entrega resultados estructurados. Estos resultados pueden ser usados por una interfaz web, por un servicio de consulta o por un proceso posterior de validación.

En este punto del desarrollo, la palabra localización se interpreta de forma limitada. El módulo no calcula una posición geográfica exacta de la emisión ni realiza triangulación con varios nodos. La relación espacial disponible se maneja a través de filtros administrativos, como código DANE, lista de DANEs o municipio, los cuales permiten contextualizar una medición dentro de una zona de análisis. El desarrollo detallado de esta parte queda para la continuación de la sección.

0.7.3. Arquitectura del módulo de postprocesamiento

El desarrollo se organiza como un módulo de Python separado en archivos con responsabilidades específicas. Esta división facilita el mantenimiento, la prueba por componentes y la integración posterior con una plataforma web o con una API.

Tabla 8: Archivos principales del módulo de postprocesamiento espectral.

Archivo	Función dentro del módulo
<code>main.py</code>	Permite ejecutar el procesamiento desde consola. Lee un archivo JSON o una trama en línea, interpreta argumentos como picos, cumplimiento, DANE, umbral y rutas auxiliares, y entrega una respuesta estructurada.
<code>server_flask.py</code>	Expone el procesamiento mediante una API basada en Flask. Permite recibir solicitudes HTTP con una captura espectral y parámetros de análisis.
<code>src/payload_parser.py</code>	Convierte el contenido del JSON de entrada en un objeto espectral interno. Valida que existan las muestras y los límites de frecuencia necesarios.
<code>src/spectrum_frame.py</code>	Define la estructura <code>SpectrumFrame</code> , que contiene amplitudes, frecuencia inicial, frecuencia final, eje de frecuencias y resolución espectral.
<code>src/processor.py</code>	Orquesta el flujo principal. Normaliza la entrada, selecciona el modo de operación, aplica corrección si existe, llama a los algoritmos de detección y arma la salida final.
<code>src/spectral_analysis.py</code>	Contiene las funciones de análisis espectral, detección de picos, segmentación de emisiones y estimación de parámetros como frecuencia central, ancho de banda, potencia, MER y BER cuando aplica.
<code>src/calibration_io.py</code>	Contiene utilidades para trabajar con archivos auxiliares, como bases de licencias o información de comparación. En este avance solo se menciona su papel dentro de la arquitectura.
<code>src/power_utils.py</code>	Contiene funciones asociadas al cálculo robusto de potencia. Integra la energía de la emisión en dominio lineal y evita reportar valores no finitos cuando la captura no permite una estimación confiable.

La arquitectura permite ejecutar el mismo procesamiento desde dos frentes: por consola, usando `main.py`, o como servicio, usando `server_flask.py`. En ambos casos, la función central es `process_input`, definida en `src/processor.py`, porque allí se decide el modo de análisis y se construye la respuesta final.

0.7.4. Flujo general de procesamiento

El flujo de trabajo inicia con una captura espectral en formato JSON. El sistema espera una lista de valores espectrales y dos frecuencias límite que definan el intervalo analizado. A partir de esa información, se construye una representación interna de la traza y se ejecuta el modo de operación correspondiente.

De forma resumida, el flujo es el siguiente:

1. Recepción del JSON espectral o de una trama equivalente.
2. Lectura de los campos principales de frecuencia y amplitud.
3. Construcción del objeto `SpectrumFrame`.
4. Aplicación opcional de corrección de ganancia, si se entrega un archivo de corrección.
5. Selección del modo de operación según los picos recibidos y el valor de cumplimiento.
6. Cálculo del umbral de detección o uso del umbral configurado por el usuario.
7. Búsqueda de picos o regiones candidatas de emisión.
8. Estimación de la frecuencia central de cada emisión candidata.
9. Estimación del ancho de banda mediante ancho a $-x$ dB u OBW cuando está disponible.
10. Estimación de potencia integrada de la emisión en la región medida.
11. Estimación opcional de MER/BER cuando la señal analizada corresponde a un caso TDT compatible.
12. Generación de una salida estructurada con el modo usado y los resultados medidos.

Este flujo conserva una separación clara entre los datos de entrada, la representación interna y los resultados. Esa decisión es importante porque evita que cada parte del sistema tenga que interpretar directamente el JSON original.

0.7.5. Entradas del sistema

Las entradas principales del módulo son la captura espectral y algunos parámetros de control. No todos los campos son obligatorios en todos los modos; algunos se usan solo cuando se quiere hacer análisis por picos o preparar una validación posterior de cumplimiento.

Tabla 9: Entradas principales consideradas en este avance del módulo.

Entrada	Tipo esperado	Descripción
<code>Pxx</code>	Lista numérica	Vector de amplitudes o potencias espectrales de la captura. Es la señal base que se analiza.
<code>start_freq_hz</code>	Numérico	Frecuencia inicial de la captura, expresada en Hz.
<code>end_freq_hz</code>	Numérico	Frecuencia final de la captura, expresada en Hz.
<code>picos</code>	Lista numérica	Frecuencias específicas solicitadas por el usuario. Si se entregan, el sistema entra al modo de análisis por picos.
<code>cumplimiento</code>	Entero 0/1	Bandera que indica si se desea ejecutar el modo de cumplimiento cuando no se entregan picos.
<code>corr</code>	Ruta de archivo	Archivo opcional de corrección de ganancia. Permite ajustar la traza antes del análisis.
<code>lic</code>	Ruta de archivo	Archivo opcional de licencias. En este avance solo se registra como dependencia del modo de cumplimiento.
<code>dane,</code> <code>municipio</code> <code>danos,</code> <code>umbral_db</code>	Texto o lista	Filtros administrativos que permiten contextualizar la captura en una zona de análisis.
	Numérico opcional	Umbral en dB sobre el piso de ruido usado para decidir si una región puede considerarse candidata de emisión.
<code>delta_fc_khz</code>	Numérico opcional	Tolerancia de frecuencia central, en kHz, usada para comparar o emparejar frecuencias cercanas.
<code>delta_bw_khz</code>	Numérico opcional	Tolerancia de ancho de banda, en kHz. En este avance se menciona como parámetro disponible, pero no se desarrolla la evaluación normativa completa.

El parser de entrada también acepta algunos alias de nombres para los campos principales, como `pxx`, `PSD`, `f_start_hz` o `f_stop_hz`. Esta flexibilidad permite integrar capturas provenientes de fuentes distintas sin cambiar la estructura general del procesamiento.

0.7.6. Salidas del sistema

La salida del módulo se entrega como una estructura de datos con información general del procesamiento y una lista de resultados. En este avance se documentan las salidas relacionadas con modo, picos, estado de detección, frecuencia central, ancho de banda, potencia y MER/BER cuando el caso aplica.

Tabla 10: Salidas generales del procesamiento.

Salida	Tipo esperado	Descripción
mode	Texto	Modo de operación seleccionado: <code>all_emissions</code> , <code>peaks</code> o <code>compliance</code> .
cumplimiento	Entero 0/1	Valor de cumplimiento recibido en la entrada.
picos_count	Entero	Cantidad de picos entregados por el usuario.
picos	Lista numérica	Picos solicitados originalmente, cuando existen.
umbral	Numérico	Umbral absoluto usado para apoyar la detección.
num_emissions	Entero	Cantidad de emisiones o candidatos encontrados.
results	Lista de objetos	Resultados individuales de cada emisión, pico solicitado o candidato analizado.

Tabla 11: Campos principales dentro de cada resultado.

Campo	Tipo esperado	Descripción
status	Texto	Indica si se encontró una emisión o si el punto analizado se consideró ruido.
reason	Texto	Explicación breve cuando no se detecta una emisión válida.
requested_pico	Numérico	Pico solicitado por el usuario en modo de análisis por picos.
matched_peak_hz	Numérico	Pico detectado más cercano al pico solicitado.

Campo	Tipo esperado	Descripción
<code>delta_match_hz</code>	Numérico	Diferencia entre el pico solicitado y el pico detectado asociado.
<code>fc_hz</code>	Numérico	Frecuencia central estimada de la emisión, expresada en Hz.
<code>fc_mhz</code>	Numérico	Frecuencia central estimada de la emisión, expresada en MHz.
<code>bw_hz</code>	Numérico	Ancho de banda reportado para la emisión, expresado en Hz.
<code>bw_khz</code>	Numérico	Ancho de banda reportado para la emisión, expresado en kHz.
<code>power_dbm</code>	Numérico o nulo	Potencia estimada de la emisión. Si el cálculo no es finito, no se fuerza un valor artificial.
<code>snr_db</code>	Numérico opcional	Relación señal a ruido estimada cuando la función de medición la entrega.
<code>mer_db</code>	Numérico opcional	MER estimado para señales compatibles, especialmente TDT. Solo aparece si el cálculo aplica.
<code>ber_est</code>	Numérico opcional	BER estimado para señales compatibles, especialmente TDT. Solo aparece si el cálculo aplica.

La estructura de salida permite que otro módulo consuma los resultados sin depender de mensajes impresos en consola. Esto es necesario para integrarlo con una plataforma web, una base de datos o una capa de visualización.

0.7.7. Modos de operación

El módulo selecciona automáticamente el modo de operación usando dos elementos de entrada: la lista de picos y la bandera de cumplimiento. La regla implementada es simple: si hay picos, se prioriza el análisis por picos; si no hay picos y cumplimiento vale uno, se activa el modo de cumplimiento; si no hay picos y cumplimiento vale cero, se detectan todas las emisiones candidatas.

Tabla 12: Modos de operación del módulo de postprocesamiento.

Modo	Condición de entrada	Uso principal
<code>all_emissions</code>	No se entregan picos y <code>cumplimiento = 0</code> .	Detectar automáticamente emisiones candidatas dentro de toda la captura espectral y reportar sus primeros parámetros técnicos.
<code>peaks</code>	Se entrega una lista de picos, sin importar el valor de cumplimiento.	Analizar frecuencias específicas solicitadas por el usuario y determinar si cerca de ellas existe una emisión o si el punto se comporta como ruido.
<code>compliance</code>	No se entregan picos y <code>cumplimiento = 1</code> .	Preparar el procesamiento para comparar emisiones medidas contra información de referencia. En este avance se llega hasta la medición técnica de la emisión y la estimación MER/BER si aplica, sin desarrollar todavía el análisis completo de licencias.

Detección automática de emisiones

En el modo `all_emissions`, el sistema analiza toda la traza espectral. Primero estima un umbral de detección a partir del piso de ruido o del valor indicado por el usuario mediante `umbral_db`. Después busca picos o regiones candidatas y, para cada una, mide frecuencia central, ancho de banda y potencia. Si la emisión corresponde a un caso TDT compatible, también puede estimar MER y BER.

Este modo es útil cuando el usuario no sabe previamente en qué frecuencias existen emisiones de interés. La salida principal de esta etapa es una lista de candidatos con su estado y sus parámetros espectrales iniciales.

Análisis por picos solicitados

En el modo `peaks`, el usuario entrega una o varias frecuencias de interés. El sistema ubica la frecuencia solicitada dentro del eje de la captura y busca un pico cercano que pueda representar una emisión real. Si encuentra una región suficientemente consistente, reporta los parámetros medidos de la emisión. Si no encuentra evidencia suficiente, el resultado se marca como ruido o como pico no asociado a una emisión clara.

Este modo es conveniente cuando se desea comprobar una frecuencia puntual, por ejemplo

una frecuencia observada visualmente en una gráfica o una frecuencia que se sospecha activa.

Modo de cumplimiento

En el modo **compliance**, el sistema queda preparado para relacionar las emisiones detectadas con una base de referencia, normalmente una base de licencias. Para entrar en este modo no deben entregarse picos y la bandera **cumplimiento** debe estar activa.

En este avance no se desarrolla la comparación completa de cumplimiento. Solo se documenta que el modo existe dentro del enrutamiento del software y que utiliza la misma base inicial de detección espectral para obtener los parámetros medidos que luego pueden ser comparados con valores nominales.

0.7.8. Análisis espectral realizado: frecuencia central, ancho de banda, potencia y MER/BER

El análisis espectral del módulo se centra en convertir una región detectada de la traza en parámetros técnicos que puedan ser interpretados por la plataforma. En este avance se documentan cuatro salidas principales: frecuencia central, ancho de banda, potencia y, solo cuando la captura lo permite, MER/BER. Estos valores no se escriben de forma manual, sino que se derivan de la región espectral detectada por el algoritmo.

Frecuencia central

La frecuencia central representa la frecuencia principal o representativa de una emisión detectada dentro de la captura. En el módulo, esta frecuencia se entrega principalmente en dos unidades: Hz mediante `fc_hz` y MHz mediante `fc_mhz`.

El proceso inicia con el eje de frecuencias generado por `SpectrumFrame`. Si la captura contiene N muestras, el objeto construye un vector de frecuencias entre `start_freq_hz` y `end_freq_hz`. Con ese eje, cada muestra de amplitud queda asociada a una frecuencia específica.

Cuando el sistema detecta un pico candidato, se toma una frecuencia inicial de referencia desde el eje de frecuencias. Luego se delimita una región de análisis alrededor del candidato y se llama a las funciones de medición espectral. El resultado final de esa medición se almacena como frecuencia central de la emisión.

En términos operativos, la frecuencia central puede provenir de tres situaciones:

- En detección automática, se obtiene a partir del pico o segmento candidato encontrado por el algoritmo.

- En análisis por picos, se calcula a partir de la emisión real encontrada cerca de la frecuencia solicitada.
- En modo de cumplimiento, se conserva como frecuencia central medida para que posteriormente pueda compararse con una frecuencia nominal.

La frecuencia central no debe interpretarse simplemente como el punto más alto de la traza. En algunas capturas, especialmente cuando hay señales anchas o regiones con ruido, el punto de mayor amplitud puede no representar de forma adecuada el centro de la emisión. Por eso el módulo no solo toma un valor visual, sino que usa la región detectada para estimar una frecuencia más representativa.

Tabla 13: Campos asociados a la frecuencia central.

Campo	Unidad	Interpretación
<code>fc_hz</code>	Hz	Frecuencia central estimada en la unidad base usada internamente por el procesamiento.
<code>fc_mhz</code>	MHz	Versión escalada de <code>fc_hz</code> , más cómoda para reportes y lectura humana.
<code>matched_peak_hz</code>	Hz	Frecuencia de pico asociada a una solicitud del usuario en modo peaks . Puede servir como referencia inicial, pero no siempre reemplaza la frecuencia central estimada.
<code>delta_match_hz</code>	Hz	Diferencia entre la frecuencia solicitada y el pico asociado. Ayuda a evaluar si el emparejamiento fue cercano.

Ancho de banda

El ancho de banda describe la extensión espectral ocupada por una emisión. En el software, este valor se obtiene después de delimitar la región asociada al pico o segmento candidato. La función de medición calcula dos referencias: un ancho a $-x$ dB respecto al pico y un ancho de banda ocupado u OBW, que representa el intervalo que contiene un porcentaje definido de la potencia de la emisión.

Para el reporte final, el procesador selecciona el ancho de banda más conveniente mediante una regla conservadora: si existe OBW válido, se prefiere este valor porque suele ser más estable en señales con mesetas, varios picos o modulaciones anchas. Si el OBW no está disponible, el sistema usa el ancho a $-x$ dB. Esta decisión evita depender de un solo bin o de una lectura visual de la gráfica.

El resultado se reporta como `bw_hz` y `bw_khz`. El primer campo mantiene la unidad interna del procesamiento y el segundo facilita la lectura del reporte. En modo de cumplimiento, el mismo valor medido se conserva como `bw_medido_kHz`, pero la comparación normativa completa queda para una etapa posterior de la sección.

Tabla 14: Campos asociados al ancho de banda.

Campo	Unidad	Interpretación
<code>bandwidth_xdb_hz</code>	Hz	Ancho calculado a $-x$ dB respecto al pico de la emisión.
<code>obw_hz</code>	Hz	Ancho de banda ocupado, calculado a partir de la potencia acumulada de la región.
<code>bw_hz</code>	Hz	Ancho de banda seleccionado para reportar la emisión.
<code>bw_khz</code>	kHz	Versión del ancho de banda reportado en una unidad más cómoda para lectura humana.
<code>bw_medido_kHz</code>	kHz	Nombre usado en modo de cumplimiento para conservar el ancho de banda medido.

Potencia de la emisión

La potencia se estima a partir de la región espectral asociada a la emisión. Para evitar errores de interpretación, el cálculo no se realiza directamente en dBm como si fuera una suma lineal. El procedimiento convierte las muestras al dominio lineal, integra o suma la contribución de los bins dentro de la región seleccionada y finalmente devuelve el resultado en dBm.

El módulo usa funciones de apoyo para hacer el cálculo más robusto. Por ejemplo, la función de potencia robusta permite integrar usando el ancho de bin del `SpectrumFrame` o una estimación basada en el eje de frecuencias. Si la captura no permite calcular una potencia finita, el procesador evita reportar `-inf`, `NaN` o valores artificiales, y conserva el resultado como nulo. Esto es importante porque una potencia no finita no debe interpretarse como una medición real.

En los modos `all_emissions` y `peaks`, la salida principal se reporta como `power_dbm`. En modo de cumplimiento, la misma magnitud se puede conservar como `p_medida_dbm`. En este avance se documenta la medición de potencia, pero no se desarrolla todavía la evaluación frente a potencia nominal de una licencia.

Tabla 15: Campos asociados a potencia.

Campo	Unidad	Interpretación
<code>power_dbm</code>	dBm	Potencia estimada de la emisión en los modos de detección general y análisis por picos.
<code>p_medida_dBm</code>	dBm	Nombre usado en modo de cumplimiento para conservar la potencia medida.
<code>channel_power_dbm</code>	dBm	Potencia de canal calculada durante la medición de parámetros de la emisión.
<code>snr_db</code>	dB	Relación señal a ruido estimada cuando la función de análisis la entrega.

MER y BER cuando aplica

El módulo también incluye una estimación opcional de MER y BER para señales compatibles con el caso TDT. Este cálculo no se aplica a todas las emisiones detectadas. La función primero verifica si la frecuencia central de la región cae dentro del rango TDT definido en el código y si el ancho de la banda es plausible para ese tipo de señal. Si esas condiciones no se cumplen, la función retorna un resultado vacío y el reporte no incluye campos de MER/BER.

Cuando el caso sí aplica, el sistema toma la banda medida como región de señal y usa regiones laterales o información fuera de banda como referencia de ruido. Con esa relación se estima una magnitud de calidad equivalente a MER en dB y luego se calcula un BER estimado para una modulación M-QAM, usando $M = 64$ en el flujo implementado para TDT.

Estos valores deben interpretarse como estimaciones de calidad obtenidas desde la traza espectral disponible, no como una demodulación completa de la señal. Por lo tanto, son útiles para orientar el diagnóstico de una captura TDT, pero deben validarse con mediciones especializadas si se requiere una certificación formal.

Tabla 16: Campos asociados a MER/BER cuando la estimación aplica.

Campo	Unidad	Interpretación
mer_db	dB	MER estimado de la señal compatible. Se reporta solo si la función encuentra condiciones válidas para el cálculo.
ber_est	Adimensional	BER estimado a partir de la relación de calidad calculada para la señal.
f_center_hz	Hz	Frecuencia central de la banda usada internamente para el cálculo de MER/BER.
band_start_hz	Hz	Límite inferior de la banda evaluada.
band_stop_hz	Hz	Límite superior de la banda evaluada.

0.8. Protocolos de prueba en nube y borde

Esta sección debe describir cómo se valida el sistema completo y sus componentes individuales, definiendo procedimientos de prueba, métricas, criterios de aceptación y condiciones bajo las cuales se ejecutan los experimentos.

Como placeholder, el texto actual sirve para organizar la documentación de pruebas funcionales, de rendimiento y de integración tanto en el entorno de borde como en la infraestructura desplegada en la nube.

Puntos sugeridos para completar

- Objetivos de prueba y alcance de validación.
- Escenarios experimentales en borde y en nube.
- Métricas de desempeño, confiabilidad y latencia.
- Procedimiento de ejecución y captura de evidencias.
- Resultados esperados, criterios de aceptación y hallazgos.

0.9. Integración general del sistema

Esta sección debe consolidar la visión de conjunto del sistema una vez integrados los módulos de hardware, adquisición SDR, procesamiento en borde, servicios en nube y análisis

posterior.

El placeholder permite dejar una base para explicar la secuencia operativa del sistema, los puntos de acoplamiento entre componentes, los riesgos de integración y las decisiones tomadas para lograr una operación coordinada.

Puntos sugeridos para completar

- Flujo extremo a extremo del sistema.
- Dependencias entre subsistemas y puntos de integración.
- Mecanismos de sincronización, intercambio y supervisión.
- Problemas encontrados durante la integración y su solución.
- Estado actual de madurez e interoperabilidad del sistema.

0.10. Conclusiones generales

Esta sección debe sintetizar los principales resultados técnicos del proyecto y resumir qué se logró demostrar con el diseño, la implementación, la integración y las pruebas realizadas sobre el sistema.

Como placeholder, se reserva este espacio para formular conclusiones claras, relacionadas con el cumplimiento de objetivos, las capacidades alcanzadas y las limitaciones que todavía deben considerarse en iteraciones futuras.

Puntos sugeridos para completar

- Principales logros técnicos del sistema.
- Relación entre objetivos planteados y resultados obtenidos.
- Fortalezas de la arquitectura o implementación.
- Limitaciones persistentes o aspectos no resueltos.
- Valor del documento como base para etapas posteriores.

0.11. Recomendaciones

Esta sección debe presentar sugerencias concretas para mejorar el sistema, fortalecer su documentación, ampliar su capacidad operativa o reducir riesgos identificados durante el desarrollo y la validación.

El placeholder actual deja preparada una estructura para priorizar acciones futuras en hardware, software, procesamiento, despliegue y trabajo colaborativo entre los integrantes del proyecto.

Puntos sugeridos para completar

- Mejoras prioritarias de corto plazo.
- Recomendaciones de mediano plazo para escalamiento.
- Ajustes sugeridos en hardware o adquisición de datos.
- Recomendaciones para pruebas, monitoreo y mantenimiento.
- Oportunidades de investigación o extensión futura.

0.12. Referencias

Esta sección debe reunir las fuentes técnicas, académicas y documentales utilizadas para sustentar decisiones de diseño, selección de tecnologías, métodos de análisis y procedimientos de implementación o prueba.

Como placeholder, este bloque ayuda a reservar la estructura necesaria para incorporar referencias bibliográficas consistentes y trazables, siguiendo el formato de citación que el equipo o la institución definan posteriormente.

Puntos sugeridos para completar

- Artículos, libros o reportes técnicos relevantes.
- Documentación oficial de hardware, SDR y plataformas en nube.
- Referencias sobre algoritmos de análisis espectral y localización.
- Normas, manuales o estándares aplicables.
- Formato de citación adoptado por el documento final.

Apéndice A

Técnica de desarrollo en grupo usando agentes IA

A.1. Propósito del capítulo

Este anexo tiene como objetivo documentar la metodología de trabajo colaborativo y la infraestructura técnica utilizada por el equipo de desarrollo al integrarse con agentes de Inteligencia Artificial (IA). Se busca proporcionar una guía clara sobre la instalación, configuración y mejores prácticas de *prompting* para asegurar que el uso de estas herramientas sea trazable, repetible y de alta calidad dentro del ciclo de vida del proyecto.

A.2. Contexto dentro del sistema general

En el desarrollo de sistemas complejos como el de adquisición SDR y plataformas *cloud*, el uso de agentes IA (como Gemini CLI y Copilot CLI) permite acelerar la automatización de flujos de trabajo, la revisión de código y la generación de documentación técnica. Esta técnica no reemplaza la supervisión humana, sino que actúa como un multiplicador de capacidades, integrando razonamiento multi-agente para la toma de decisiones técnicas y el análisis de contexto.

A.3. Desarrollo técnico: Ecosistema de Agentes CLI

El equipo utiliza una combinación de herramientas de terminal que permiten una interacción directa con modelos de lenguaje avanzados.

A.3.1. Instalación y Configuración

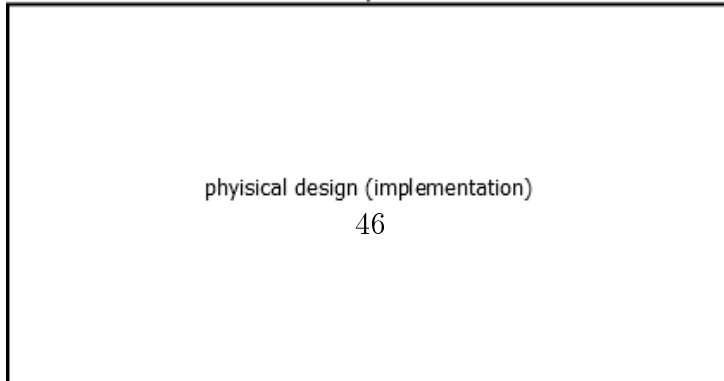
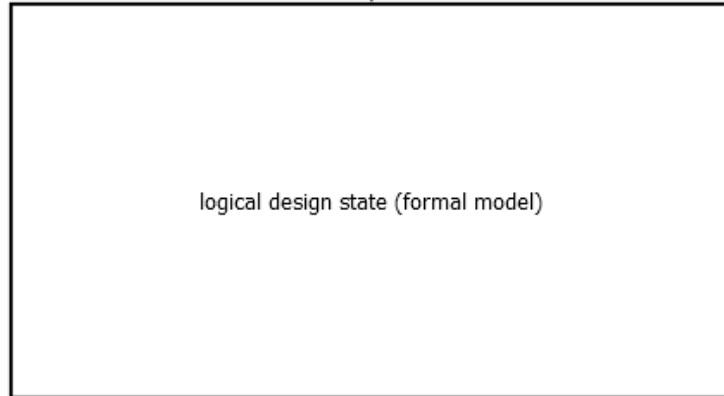
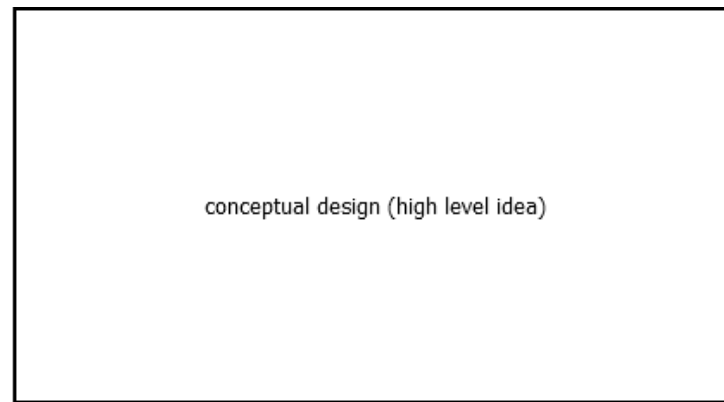
A continuación se detallan los comandos principales para habilitar el entorno de agentes:

- **GitHub Copilot CLI:** Utilizado principalmente para asistencia en codificación y automatización de tareas de repositorio.
 - Instalación: `npm install -g @github/copilot`
 - Autenticación: `copilot /login`
- **Gemini CLI:** Utilizado para tareas multimodales, gestión de habilidades (*skills*) y análisis de arquitectura.
 - Instalación: `npm install -g @google/gemini-cli`
 - Inicio: `gemini`

A.3.2. Arquitectura Multi-Agente

El sistema se diseña bajo un esquema de roles especializados que colaboran para resolver tareas complejas:

- **Agente Coordinador:** Asigna roles y selecciona el modelo más apto.
- **Agente de Interpretación:** Analiza el contexto de la petición.
- **Agente de Backend:** Gestiona el almacenamiento y la comunicación con APIs.
- **Agente de Procesamiento:** Maneja tareas específicas de señales (audio/espectro).



A.4. Decisiones de diseño: Metodología de Prompting

Para minimizar alucinaciones y maximizar la coherencia de las respuestas, se ha adoptado una estructura obligatoria para cada interacción con los agentes, basada en el modelo **ROLE** → **OBJECTIVE** → **CONTEXT** → **REASONING**.

Tabla A.1: Componentes de un Prompt de alta calidad.

Componente	Descripción
Role	Define quién debe responder (e.g., "Senior Software Developer").
Objective	Define qué se quiere lograr de forma clara y concisa.
Context	Proporciona archivos relevantes, restricciones y entorno.
Reasoning	Solicita al agente explicar su lógica antes de entregar el resultado.
Output Format	Especifica el formato deseado (JSON, LaTeX, C code, etc.).

A.5. Problemas presentados y ajustes realizados

Durante la implementación de esta metodología, se identificaron los siguientes desafíos:

Tabla A.2: Problemas en la colaboración con IA y ajustes aplicados.

Problema	Evidencia	Causa probable	Decisión o Resultado
Alucinaciones técnicas	Código que usa librerías inexistentes.	Contexto insuficiente en el prompt inicial.	Implementar el patrón ROLE-CONTEXT .
Pérdida de hilo conductor	El agente olvidó restricciones previas.	Ventana de contexto saturada.	Uso de comandos /compact y archivos .geminiignore .

A.6. Validación y Evidencias

La metodología se valida mediante la revisión humana de cada entrega. El flujo de trabajo colaborativo asegura que ningún cambio se integre sin pasar por un ciclo de validación

cruzada entre agentes y desarrolladores.

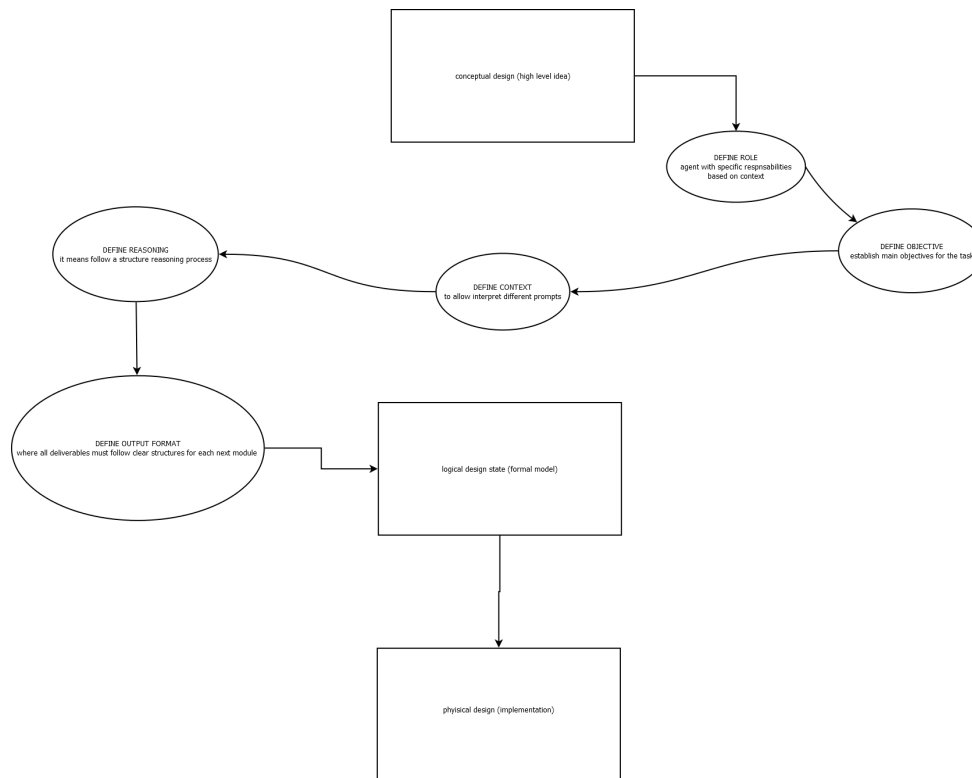


Figura A.2: Roles asignados y estados de diseño en el flujo de trabajo IA. Fuente: Elaboración propia.

A.7. Aprendizajes y transferencia de conocimiento

- **Criterio de Realidad:** La IA es un asistente, no un ejecutor autónomo sin supervisión. La validación humana es mandatoria.
- **Modulariad:** Es preferible pedir tareas pequeñas y específicas que una solución completa y compleja en un solo prompt.
- **Documentación:** Mantener archivos `GEMINI.md` o `AGENTS_CLI.md` en la raíz del proyecto ayuda a los agentes a entender rápidamente las convenciones del equipo.

A.8. Recomendaciones específicas

Se recomienda la rotación de roles entre los desarrolladores para familiarizarse con las capacidades de diferentes *backends* (GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro) según

la naturaleza de la tarea.

A.9. Diagramas adicionales

Este anexo está destinado a reunir diagramas complementarios que apoyen la comprensión del sistema, pero que por extensión o nivel de detalle no convenga ubicar en el cuerpo principal del documento.

Como placeholder, aquí pueden integrarse esquemas de arquitectura, diagramas de bloques, secuencias de operación, topologías de red o representaciones de flujo de datos más detalladas que las incluidas en las secciones centrales.

Puntos sugeridos para completar

- Diagramas de arquitectura extendida.
- Diagramas de despliegue o conectividad.
- Secuencias operativas o flujos de eventos.
- Esquemas detallados de hardware o adquisición.
- Figuras de apoyo para pruebas o integración.

A.10. Fragmentos de código relevantes

Este anexo debe agrupar fragmentos de código que ayuden a ilustrar decisiones de implementación, configuraciones importantes o partes clave del sistema sin sobrecargar el desarrollo principal del informe.

El placeholder actual reserva una estructura útil para incluir ejemplos de scripts, configuraciones, endpoints, pipelines de procesamiento o comandos de despliegue con su respectiva explicación técnica.

Puntos sugeridos para completar

- Código de adquisición o preprocesamiento en borde.
- Fragmentos de servicios, APIs o componentes web.
- Scripts de automatización o despliegue.

- Configuraciones técnicas relevantes para reproducibilidad.
- Comentarios sobre decisiones de implementación.

A.11. Manuales o guías de uso

Este anexo debe compilar instrucciones operativas para instalar, ejecutar, mantener o demostrar el sistema, orientadas a integrantes del equipo, evaluadores o futuros usuarios que necesiten reproducir el entorno de trabajo.

Como placeholder, este archivo deja definido el espacio para documentar procedimientos paso a paso, dependencias, configuraciones previas y recomendaciones de uso seguro del sistema.

Puntos sugeridos para completar

- Requisitos previos de instalación.
- Procedimiento de puesta en marcha.
- Instrucciones de operación cotidiana.
- Solución de problemas frecuentes.
- Recomendaciones para mantenimiento y actualización.

A.12. Resultados de pruebas

Este anexo debe almacenar tablas, capturas, mediciones y evidencias extendidas derivadas de las pruebas ejecutadas sobre el sistema, especialmente cuando el nivel de detalle excede lo conveniente para el cuerpo principal del documento.

El placeholder actual deja lista una sección para organizar resultados cuantitativos y cualitativos, comparaciones entre escenarios y observaciones complementarias sobre el comportamiento del sistema durante la validación.

Puntos sugeridos para completar

- Tablas de métricas y mediciones crudas.
- Capturas o evidencias visuales de ejecución.

- Comparación entre escenarios de prueba.
- Observaciones adicionales y anomalías detectadas.
- Trazabilidad entre resultados y protocolos aplicados.